

CSOPESY -- 3 Threads and Concurrency

Overview

- Threads run within application
- Basic unit of CPU utilization
 - thread id
 - program counter
 - register set
 - and a stack
- Multiple tasks with the application can be implemented by separate threads
 - update display
 - fetch data
 - spell checking
 - answer a network request

Process creation is heavy-weight while thread creation is light weight

- Can simplify code, increase efficiency
- kernels are generally multi-threaded

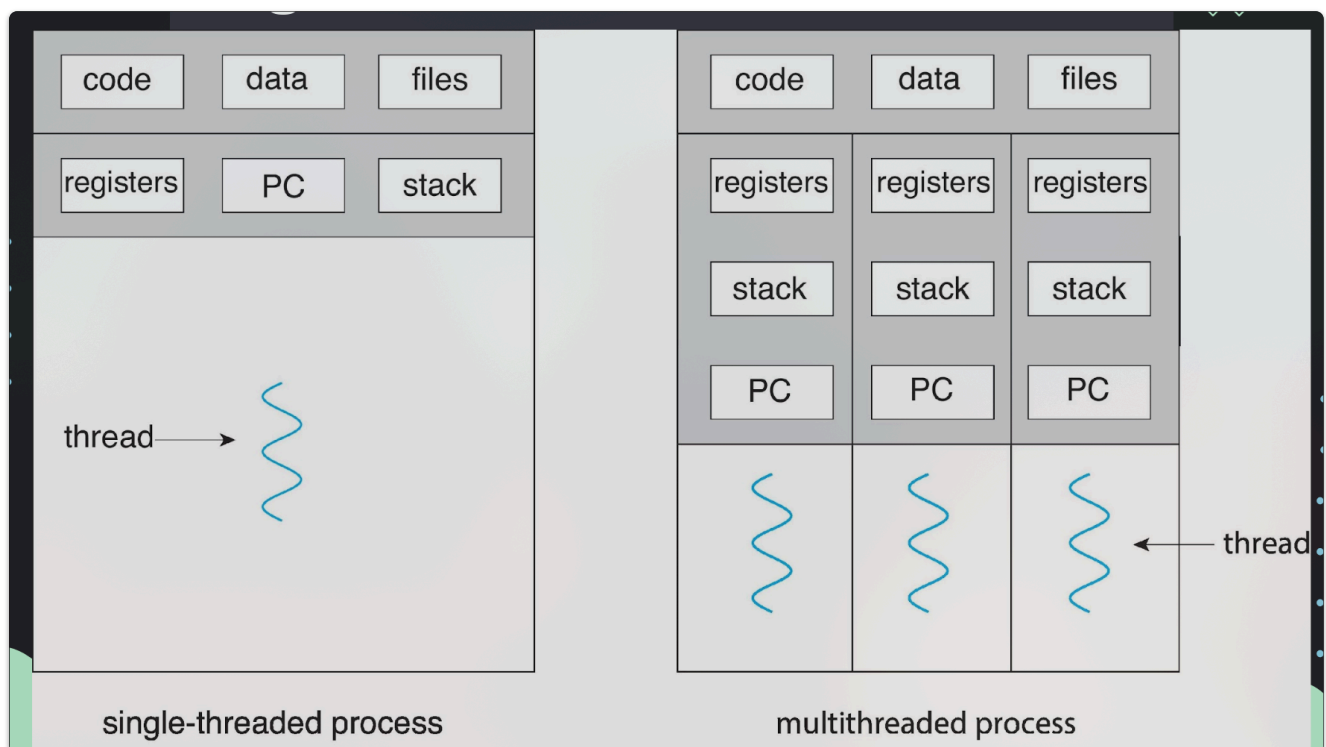


Image 1: Single thread vs Multithread

Most software applications on modern computers are multithreaded.

Applications can also be designed to leverage processing capabilities on multicore systems. In certain situations, single application maybe required to perform several tasks (i.e., web server accepts client requests for web pages, images, sound...)

Implication: if web server was single-threaded, it could service only one client at a time, it would take a long time for the process to complete because clients have to wait for a long time.

Solution:

Server run as a single process that accepts request then create a separate process for that request. (Process-creation method, see [CSOPESY -- 2 Processes > Process Creation](#)). Process creation is time-consuming and resource intensive. But if new process will do the same task, why have the overhead?

When a request is made, instead of creating another process, the server just creates a new thread to service the request within the process and resumes listening for additional requests.

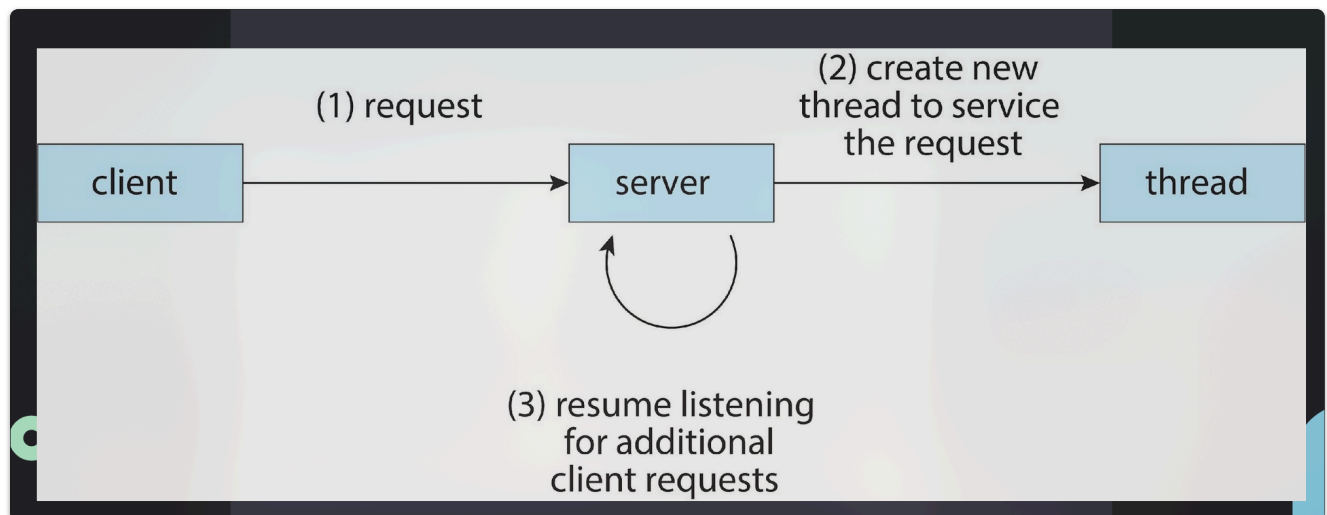


Image 2: Multithreading architecture

Most OS kernels are also multithreaded, Linux is (I use arch btw) multithreaded. I.e., on startup boot time (I use GRUB bootloader btw), several kernel threads are created.

Many applications take advantage of multiple threads like basic sorting, trees, and graph algos.

Benefits

1. Responsiveness

- may allow execution if part of process is blocked, especially important for user interfaces

2. Resource Sharing

- threads share resources of process, easier than shared memory or message passing

3. Economy

- cheaper than process creation, thread switching overhead costs less than context switching overhead

4. Scalability

- process can take advantage of multicore architectures

Multicore Programming

Single CPU systems evolved into multi-CPU systems. Later, multiple computing cores on a single processing chip where each core appears as a separate CPU to the operating system. These systems are called multicore and multithreaded programming provides mechanism for more efficient use of multiple computing cores

Rationale:

Consider an application with 4 threads. On a system with a single computing core, concurrency (or the threads running concurrently) will just mean that the threads will just be alternating because the core is capable of executing only one thread at a time.

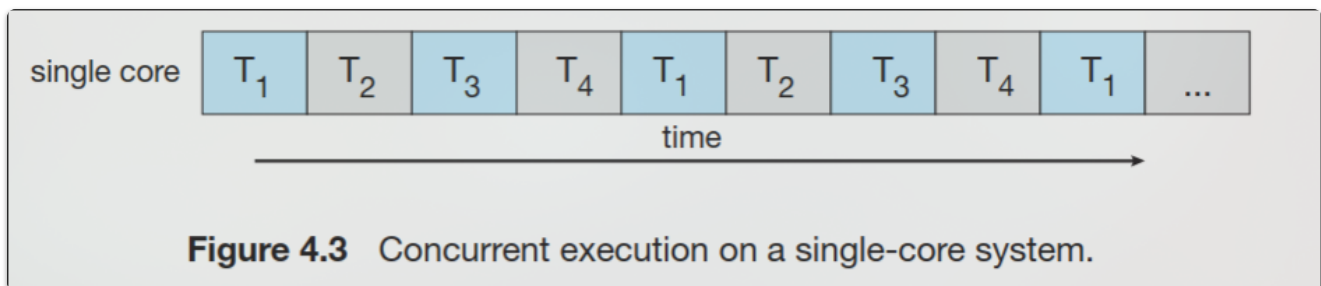


Image 3: Concurrent execution of 4 threads in single-core system over time

BUT IN A SYSTEM WITH MULTIPLE CORES (multicore), concurrency then means that some threads can run in parallel, because the system can assign a separate thread to each core

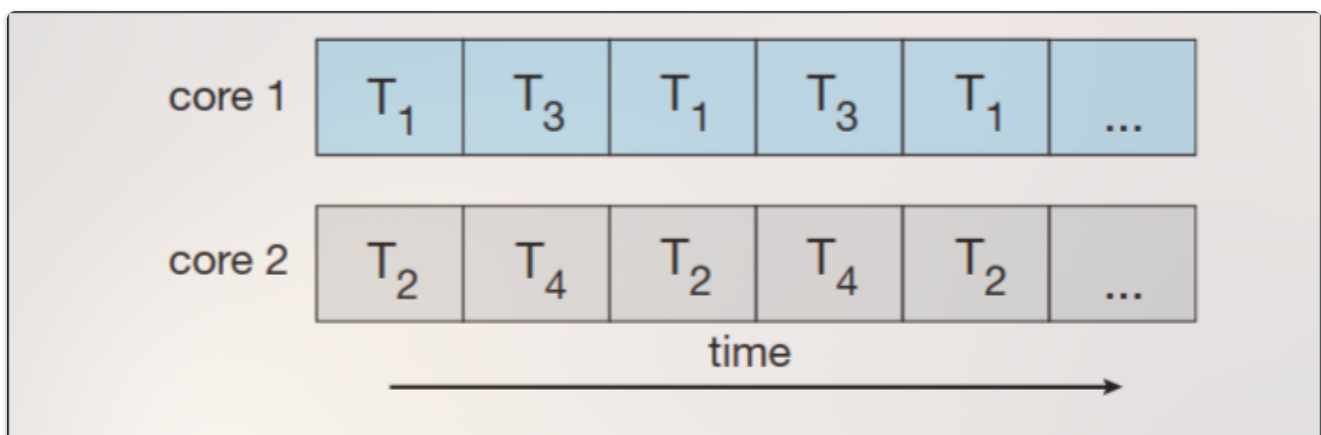


Image 4: Parallel (concurrent) execution on a multicore system

Notice distinction between **concurrency** and **parallelism**

A concurrent system supports more than one task by allowing all tasks to make progress

A parallel system can perform more than one task simultaneously.

It is possible to have concurrency without parallelism. Before multicore, CPU schedulers were designed to provide illusion of parallelism with rapid thread switching between processes.

Programming Challenges

Designers of OS must write scheduling algorithms that use multiple processing cores to allow parallel execution as **Image 4**

Five areas that present challenges

1. Identifying tasks

- examining applications to find areas that can be divided into separate but concurrent tasks

2. Balance

- while identifying tasks that can run in parallel, programmers must also ensure that the tasks perform equal work of equal value (balancing tasks). A certain task may not contribute as much to the overall process as other tasks, thus not justifying the separate core to execute

3. Data splitting

- just as applications are divided into separate tasks, data accessed and manipulated by tasks must be divided to run on separate cores

4. Data dependency

- data accessed by tasks must be examined for dependencies between two or more tasks. When one task depends on the other, programmers must ensure execution of tasks is synchronized to accommodate the data dependency

5. Testing and debugging

- when program is running in parallel on multiple cores, many different execution paths are possible. Multicore is inherently more difficult to test and debug

Types of Parallelism

1. Data Parallelism

- distributing subsets of the same data across multiple computing cores and performing same operations on each core.

2. Task Parallelism

- distributing tasks (threads) across multiple computing cores. Each thread is performing a unique operation

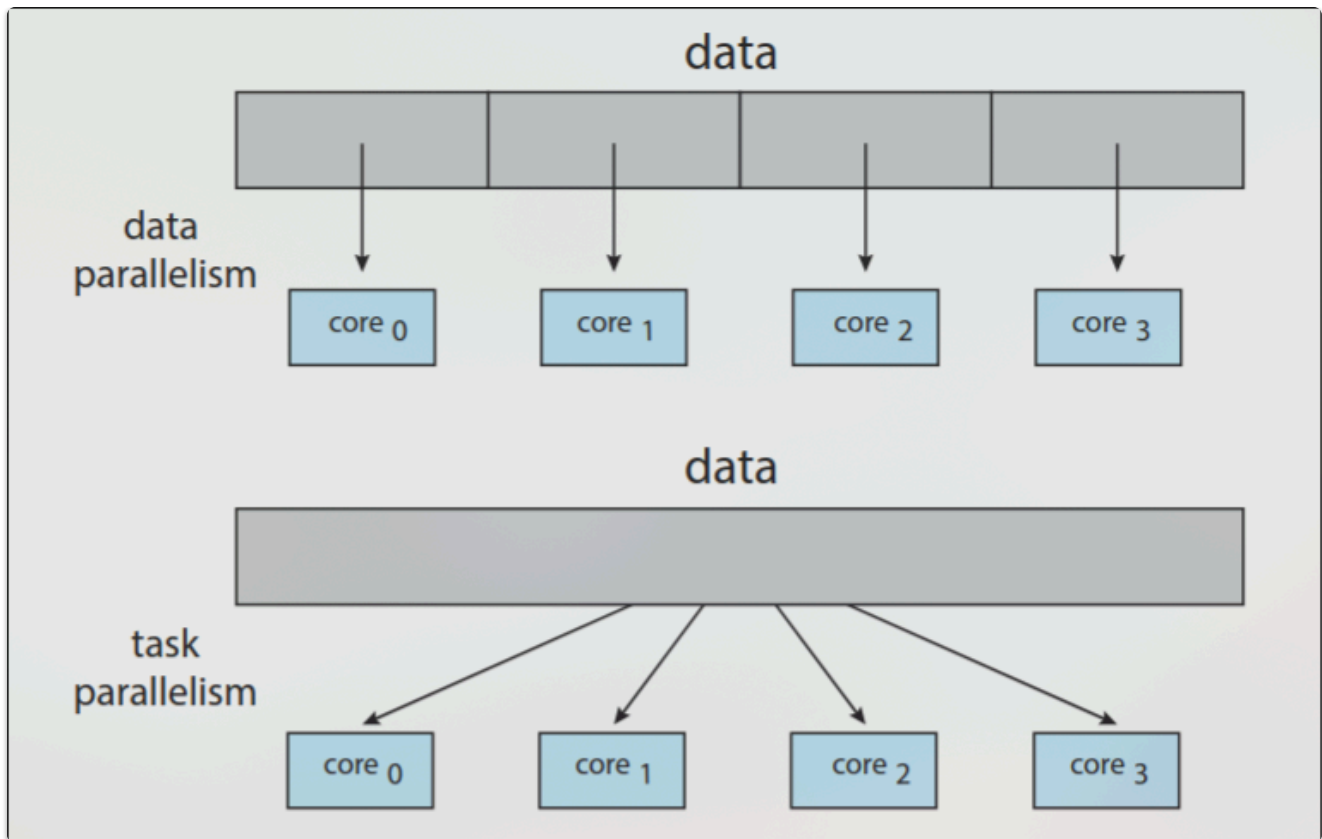


Image 5: Data and task parallelism

Amdahl's Law

Amdahl's Law

- Identifies performance gains from adding additional cores to an application that has both serial and parallel components
- S is serial portion
- N processing cores

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- That is, if application is 75% parallel / 25% serial, moving from 1 to 2 cores results in speedup of 1.6 times
- As N approaches infinity, speedup approaches 1 / S
- **Serial portion of an application has disproportionate effect on performance gained by adding additional cores**
- But does the law take into account contemporary multicore systems?

But what about if adding two additional cores for a total of four?

Then the equation will be as such:

$$\frac{1}{S + \frac{1-S}{N}} = \frac{1}{0.25 + \frac{1-0.25}{4}} = 2.28 \text{ times increase in speed}$$

User Threads and Kernels

- management done by user-level threads library
- Three primary thread libraries:
 - POSIX Pthreads
 - Windows threads
 - Java threads
- Kernel threads are supported by kernel
- Examples of OS whose kernel supports kernel threads
 - Windows
 - Linux
 - Mac OS X
 - iOS
 - Android

User threads are supported above the kernel and are managed without kernel support whereas kernel threads are support and managed directly by OS

Ultimately, a relationship must exist between user threads and kernel threads

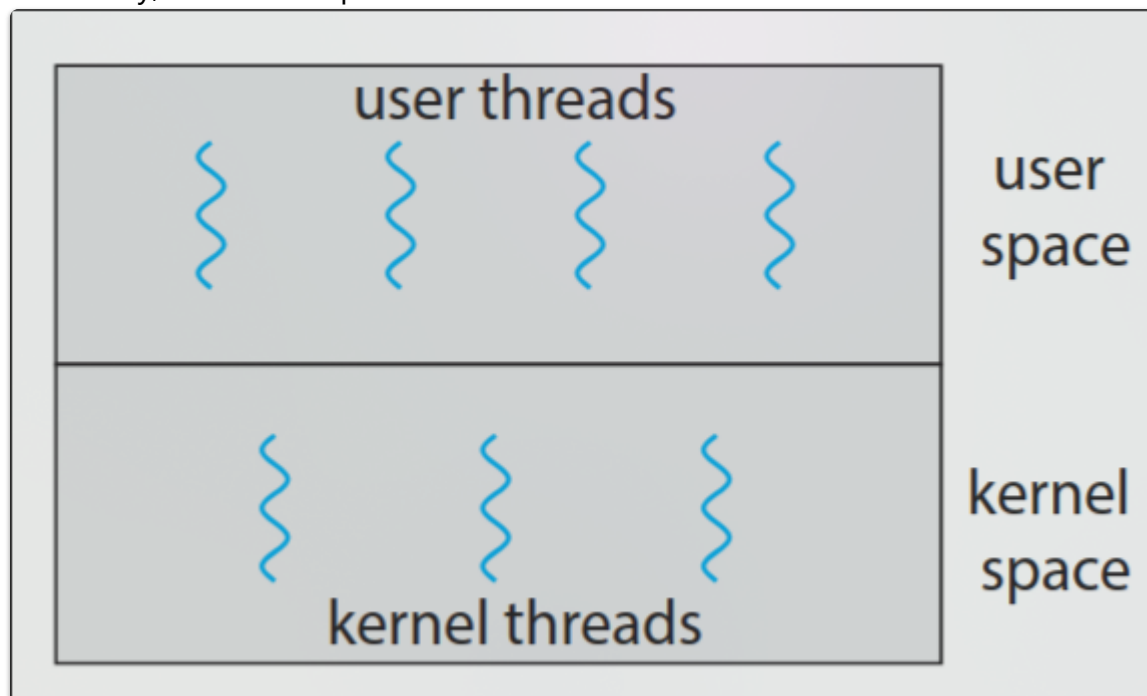


Image 6: User and Kernel Threads

To define the relationships, we use multithreading models.

Multithreading Models

1. Many to One
2. One to One
3. Many to Many

Many to One

- many user level threads to one kernel thread.
- Thread management is done by thread library in user space so it is efficient
- HOWEVER, entire process will block if one thread makes a blocking system call because only one thread can access the kernel at a time.
 - multiple threads are unable to run in parallel on multicore systems

Only few systems use this model

Example: Solaris Green Threads, GNU Portable Threads

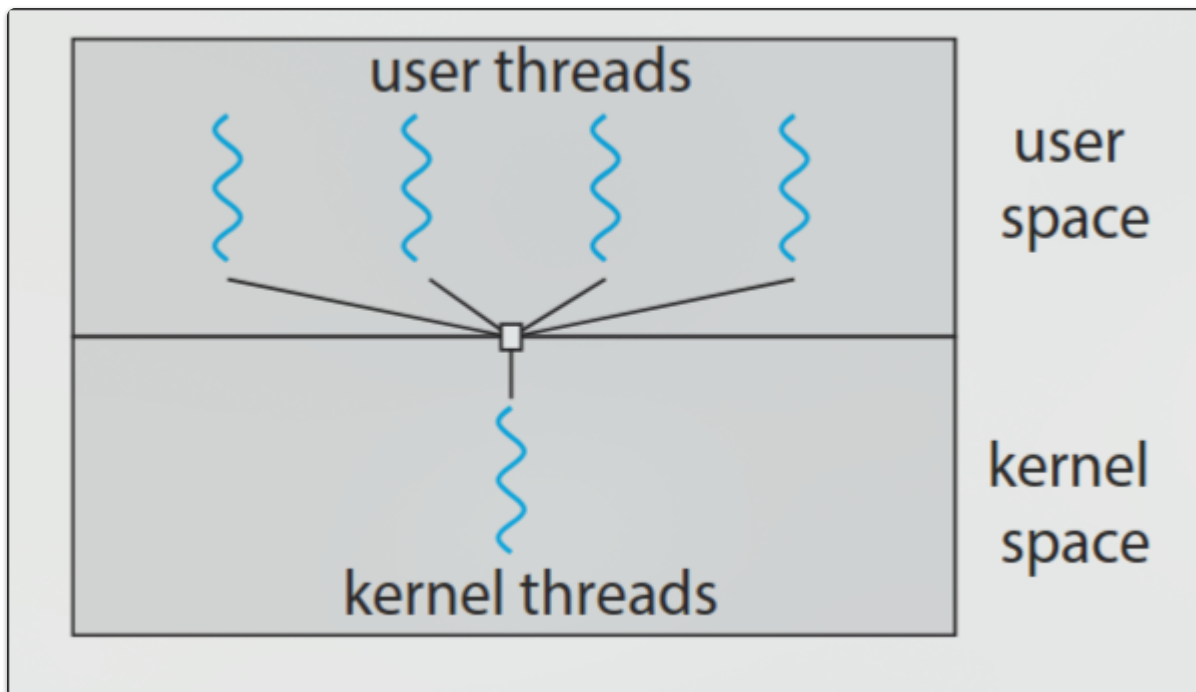


Image 6: Many to One model

One to One Model

- Maps each user thread to a kernel thread
- provides more concurrency than many to one by allowing another thread to run when a thread makes a blocking system call
- Allows multiple threads to run in parallel on multiprocessors.
- DRAWBACK: creating a user thread requires creating a corresponding kernel thread.
- Linux (using arch btw) implements One to One

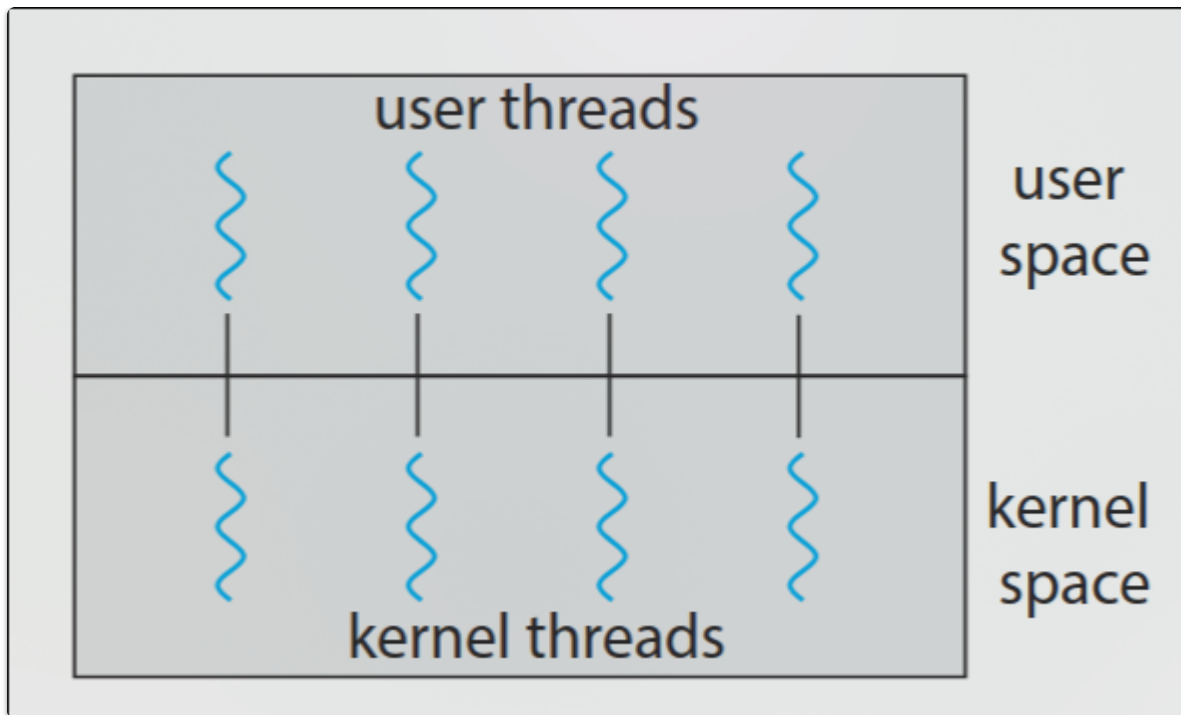


Image 7: One to One Model

Many to Many Model

- Multiplexes many user-level threads to a smaller or equal number of kernel threads
- Number of kernel threads may be specific to either a particular application or particular machine
- EXAMPLES: Java on Solaris

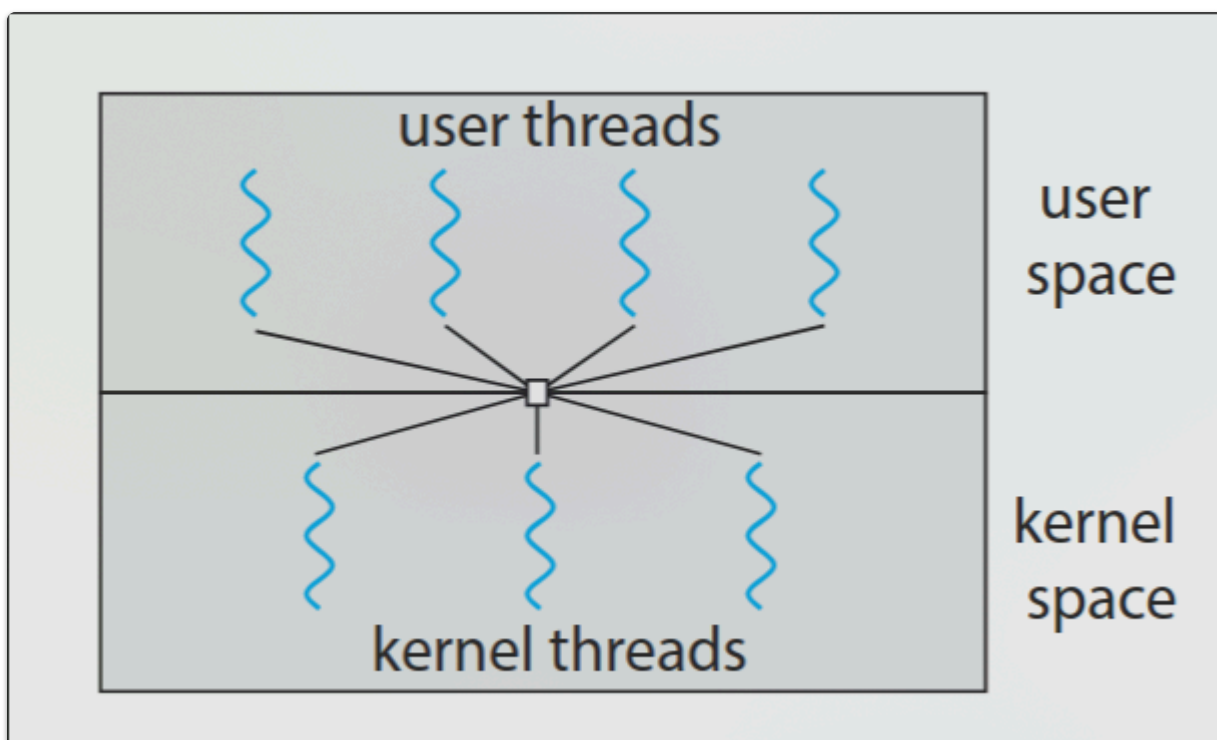


Image 8: Many to Many: Many to Many Model

There also exists a two-level model

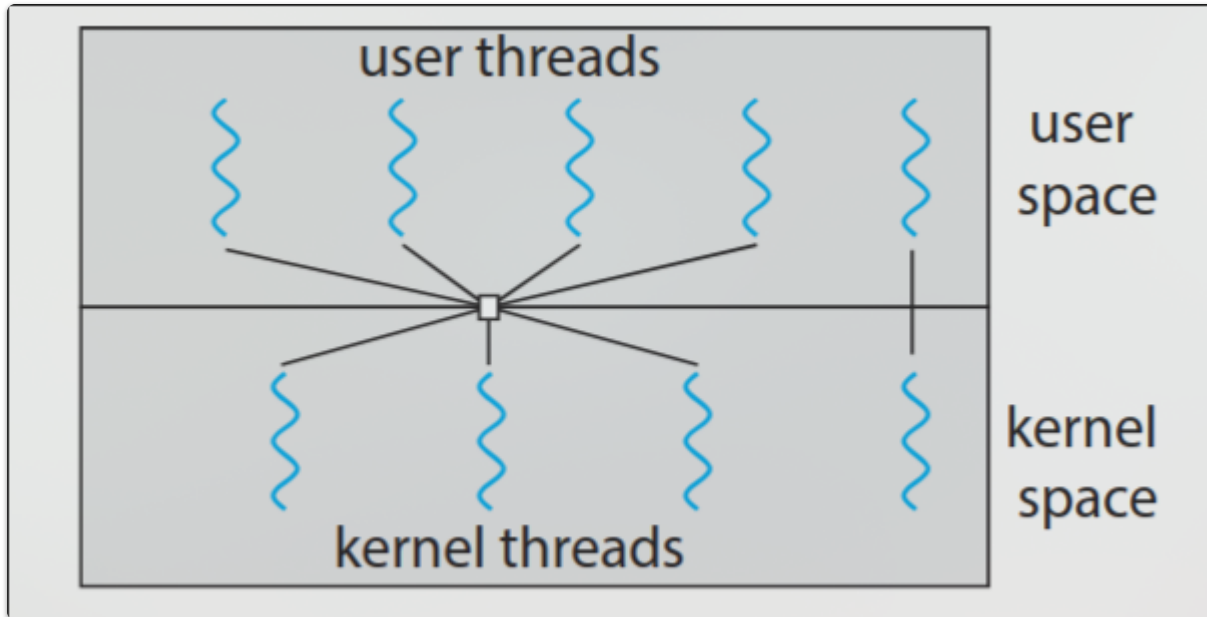


Image 9: Two-level model

Similar to Many to Many but allows user thread to be bound to kernels

POSIX Thread Library

- Either user-level or kernel-level thread
- A POSIX standard (IEEE 1003.1c) API for thread creation and synchronization
- API specifies behavior of thread library, implementation is up to development of library

```

#include <pthread.h>
#include <stdio.h>

#include <stdlib.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of thread attributes */

    /* set the default attributes of the thread */
    pthread_attr_init(&attr);
    /* create the thread */
    pthread_create(&tid, &attr, runner, argv[1]);
    /* wait for the thread to exit */
    pthread_join(tid, NULL);

    printf("sum = %d\n", sum);
}

/* The thread will execute in this function */
void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;

    for (i = 1; i <= upper; i++)
        sum += i;

    pthread_exit(0);
}

```

Image 9: C program using Pthreads API

```
#define NUM_THREADS 10

/* an array of threads to be joined upon */
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
    pthread_join(workers[i], NULL);
```

Image 10: Pthread code for joining ten threads

Check page 171 on explanation of the code

Windows Thread Library

Needs windows.h file from Windows API


```

#include <windows.h>
#include <stdio.h>
DWORD Sum; /* data is shared by the thread(s) */

/* The thread will execute in this function */
DWORD WINAPI Summation(LPVOID Param)
{
    DWORD Upper = *(DWORD*)Param;
    for (DWORD i = 1; i <= Upper; i++)
        Sum += i;
    return 0;
}

int main(int argc, char *argv[])
{
    DWORD ThreadId;
    HANDLE ThreadHandle;
    int Param;

    Param = atoi(argv[1]);
    /* create the thread */
    ThreadHandle = CreateThread(
        NULL, /* default security attributes */
        0, /* default stack size */
        Summation, /* thread function */
        &Param, /* parameter to thread function */
        0, /* default creation flags */
        &ThreadId); /* returns the thread identifier */

    /* now wait for the thread to finish */
    WaitForSingleObject(ThreadHandle, INFINITE);

    /* close the thread handle */
    CloseHandle(ThreadHandle);

    printf("sum = %d\n", Sum);
}

```

Image 11: Multithreaded C program using Windows API

Check page 172 for explanation

Thread Pools

- Create a number of threads in a pool where they await work
- this is usually slightly faster to service a request with an existing thread than create a new thread

- allows number of threads in the applications to be bound to size of pool
- tasks could be scheduled periodically

What problem/s does Thread pool solve?

- Amount of time required to create the thread, together with the fact that the thread will be discarded once it has completed its work
- if we allow each concurrent requests to be serviced in a new thread, we have not placed a bound on the number of threads concurrently active
 - Unlimited threads exhaust system resources, CPU time and memory

Thus solved by the thread pool

General Idea

- create a number of threads at start-up and place them into a pool
 - they sit and wait for work

When a server receives a request, rather than creating a thread, submit request into thread pool, resume waiting for additional requests. If there is available, awaken the thread and request is serviced immediately.

If pool contains no available no thread, queue task until a thread becomes free

The number of threads in the pool can be set heuristically based on factors such as number of CPUs in the system, the amount of physical memory and expected number of concurrent client requests

More sophisticated thread pools can dynamically adjust number of threads in the pool according to usage patterns. Such architectures provide further benefit of having a smaller pool - thereby consuming less memory - when the load on the system is low.

Fork Join

- Strategy for thread creation known as fork-join model

Main parent thread creates one or more child threads (*forks*) then said main parent thread waits for children to terminate and *join* with it, at that point can retrieve and combine results.

In implicit threading, threads are not constructed directly during fork stage; rather parallel tasks are designated

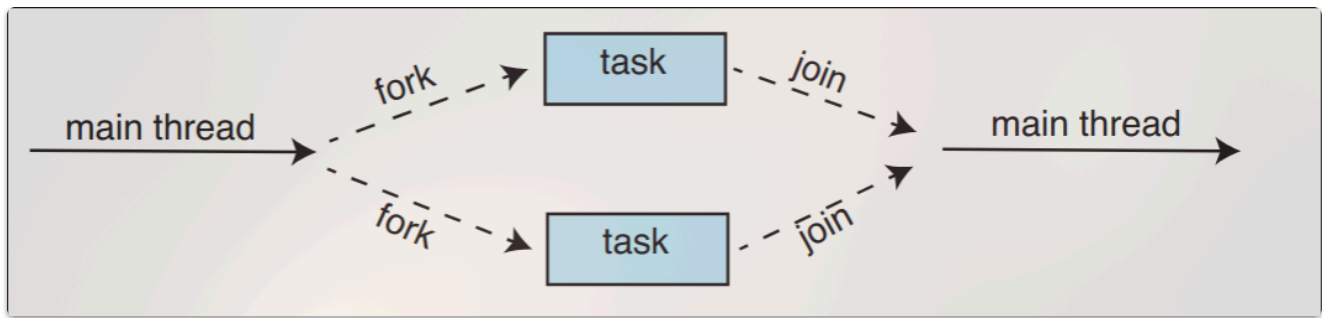


Image 11: Fork-join parallelism

Notice in **Image 11**, parallel tasks are made, but the threads are made once the said parallel task is made.

Intuition:

Recursively call a divide and conquer algorithm. Consider this code below

```
Task(problem)
  if problem is small enough
    solve the problem directly
  else
    subtask1 = fork(new Task(subset of problem)
    subtask2 = fork(new Task(subset of problem)

    result1 = join(subtask1)
    result2 = join(subtask2)

    return combined results
```

Image 12: Java code implementing fork-join model

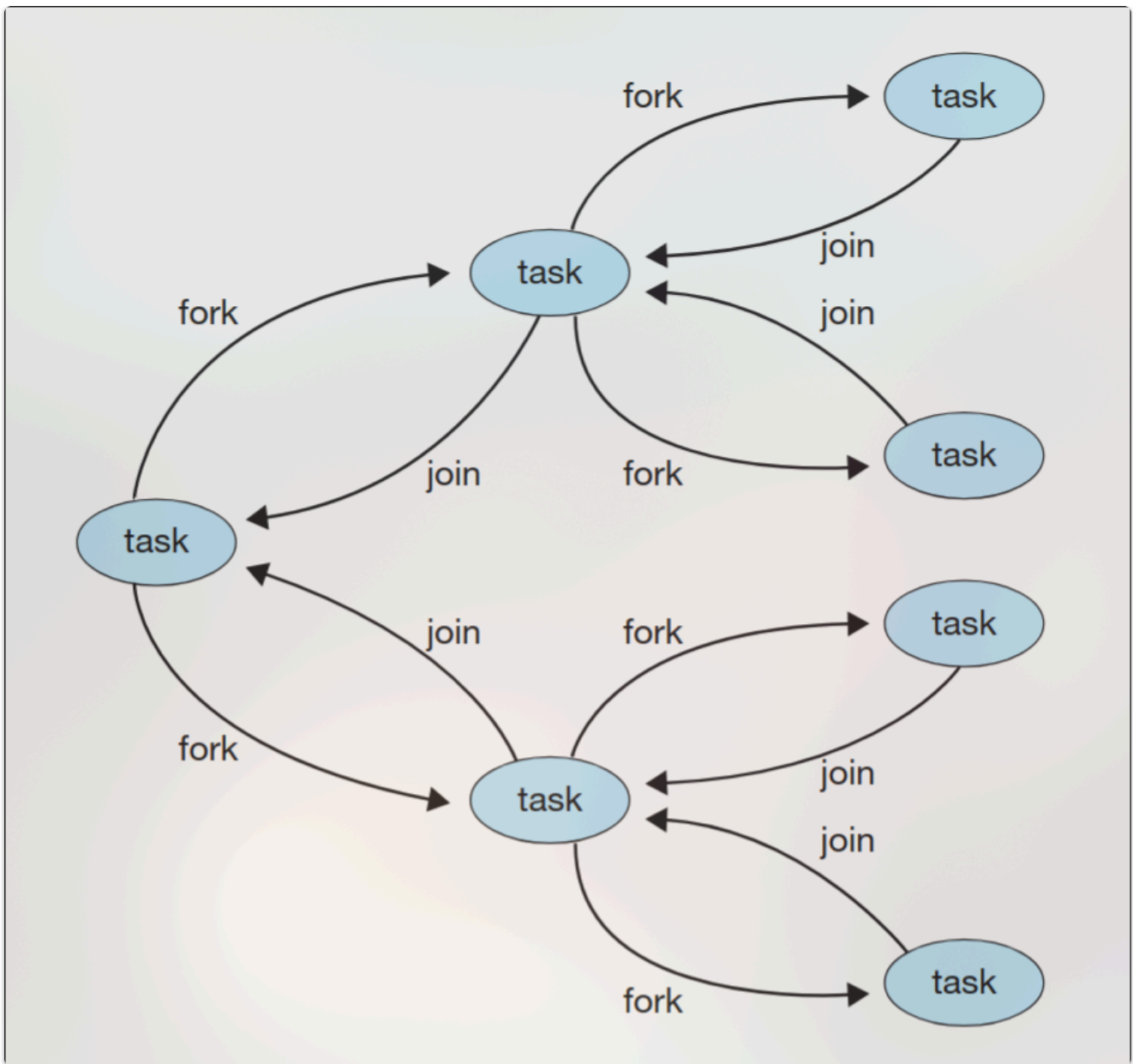


Image 13: Flowchart of the recursive function

OpenMP

- set of compiler directives, also API for programs written in C, C++, FORTRAN.

- identifies parallel regions as block of code that may run in parallel.

```
#include <omp.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    /* sequential code */

    #pragma omp parallel
    {
        printf("I am a parallel region.");
    }

    /* sequential code */

    return 0;
}
```

Image 13: Example of code that is running in parallel

When the OpenMP encounters the directive

```
#pragma omp parallel
```

it creates as many threads as there are processing cores in the system.
I.e., a dual-core will create two threads, quad-core will create 4 threads

Grand Central Dispatch

- GCD technology developed by Apple for macOS and iOS operating systems.
- a runtime library, api and language extensions that allow devs to identify sections of code (*tasks*) to run in parallel. GCD manages most of the details of threading
- allows identification of parallel sections
- Block of parallel code is in "^{}"

```
^{printf("Hello World");}
```

- blocks are placed in dispatch queue
 - GCD schedules tasks for run-time execution here

- when remove task from dispatch queue, assign task to an available thread from thread pool.
- 2 dispatch queue: **serial** and **concurrent**

Tasks in **serial** queue are removed in FIFO order (First in First Out - queue)

Each process has its own serial queue (**main queue**) and devs can create additional that are local to a particular process (**private dispatch queues**)

Tasks in **concurrent** queue, remove in FIFO order but several tasks may be removed at a time
Several system-wide concurrent queues (**global dispatch queues**), are divided into 4 primary quality of service classes:

1. QOS_CLASS_USER_INTERACTIVE

- **user-interactive**
- tasks that interact with user (UI, event handling)

2. QOS_CLASS_USER_INITIATED

- **user-initiated**
- similar to user-interactive
- associated with responsive user interface (opening a file or URL)
- do not need to be serviced as quickly as user-interactive queue

3. QOS_CLASS_UTILITY

- **utility**
- longer time to complete but do not demand immediate results
- importing data

4. QOS_CLASS_BACKGROUND

- **background**
- not visible to user and not time sensitive
- mailbox system and performing backups

Intel Thread Building Blocks

- template library for designing parallel C++ loops
- requires no special compiler or language support
- devs specify tasks that can run in parallel and TBB task scheduler maps these tasks onto underlying threads
- task scheduler provides load balancing and is cache aware.
- a developer could use TBB which provides `parallel_for` template that expects two values:

```
parallel_for (range, body)``
```

where **range** refers to **range of elements** (iteration space);

body specifies an operation that will be performed on subrange of elements (i.e., the code

block)

using this parallel for example:

```
for (int i = 0; i < n; i++){  
    apply(v[i]);  
}
```

following the TBB template, it will become

```
parallel_for (size_t(0), n (range of elements), [=](size_t i)  
{apply(v[i]);});
```

- first two params specify iteration space from 0 to n-1 which corresponds to the number of elements in array v.
- second param is C++ lambda function.

[=](size_t i) which assumes values over the iteration space (0 to n-1)
Each value of i is used to identify which array element in v to be passed as parameter to apply(v[i])

TBB library divide loop iterations into separate "chunks" and create a number of tasks that operate on those chunks

Issues on Threading

Semantics of fork and exec

- Does fork() duplicate only the calling thread or all threads?
Remember that fork() system call is used to create a separate duplicate process.

A:

- Some UNIX systems have two versions of fork()
 - one that duplicates all threads
 - another that duplicates only the thread invoked by fork() sys call
- the exec() sys call works the same way as earlier described.
if a thread invokes exec() sys call, program in parameter to exec() will replace entire process including all threads
Therefore, the specific version of fork() is dependent on application
if exec() is called immediately after forking, then duplicating all threads is unnecessary
if separate process does not call exec() after forking, then separate process should duplicate all threads

Signal Handling

- Signals used in UNIX systems to notify a process an event occurred
- may be received either synchronously or asynchronously
 - dependent on source of and reason for event being signaled

both follow the same pattern:

1. A signal is generated by occurrence of particular event
2. Signal is delivered to a process
3. once delivered, signal must be handled

Example of synch:

- illegal memory access and division by 0

synch signals are delivered to same process that performed the operation that caused the signal

When a signal is generated by an event external to running process, that signal is asynch

Example of asynch:

- terminating a process with specific keystrokes

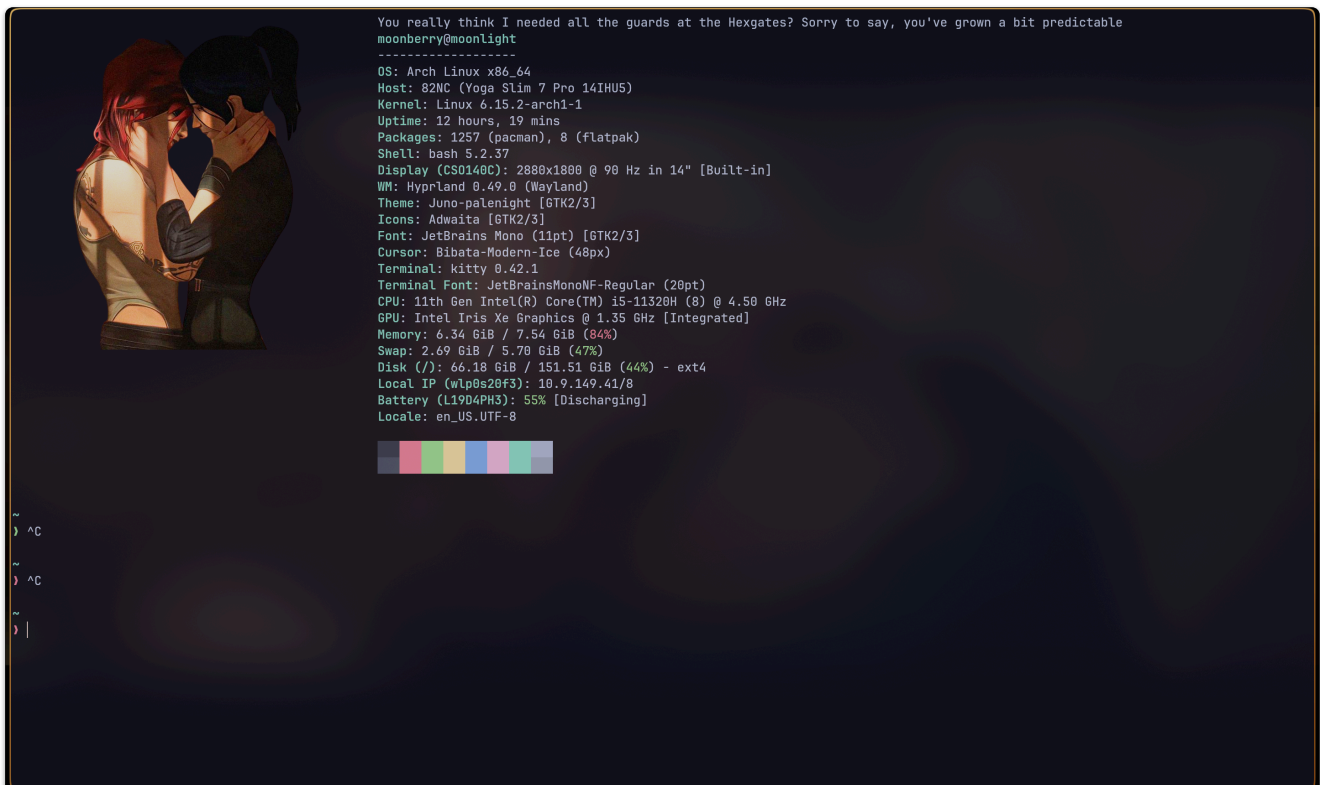


Image 14: Ctrl + C stops a process (no process right now but thats how it works in Linux, arch btw, love caivi

A signal may be **handled** by one of two handlers:

1. a default signal handler
 - every signal has a default handler that kernel runs
 2. a user-defined signal handler
 - can override default
- For single threaded applications - signal delivered to process

How about in a multi-threaded program where process may have several threads?

These options exist:

1. Deliver the signal to the thread to which the signal applies
2. Deliver the signal to every thread in the process
3. Deliver the signal to certain threads in the process
4. Assign a specific thread to receive all signals for the process

Synch signals need to be delivered to thread causing signal not other threads in process

asynch signals, not as clear. i.e., ctrl + c signal to stop a process in terminal should be sent to all threads

the standard unix function to deliver a KILLING signal is:

```
kill(pid_t pid, int signal)
```

See link [Signals - Linux manual page](#)

Most multithreaded versions of UNIX allow a thread to specify which signals it will accept and which it will block.

THEREFORE, in some cases, asynch signal may be delivered only to those threads that are not blocking it.

POSIX pthread also allows a signal to be delivered:

- pthread_kill(pthread_t tid, int signal)

Windows uses Asynch Procedure Calls (APCs)

Thread Cancellation

- terminating a thread before it has completed
 - i.e., multiple threads are concurrently searching through a DB and one thread returns the result. In this case, the other threads might be cancelled
 - Another, user presses a button on a web browser that stops web page from loading any further
 - **Target thread** - thread to be cancelled

Two scenarios:

1. Asynch cancellation - one thread immediately terminates the target thread
2. Deferred Cancellation - target thread periodically checks whether it should terminate, allowing opportunity to terminate itself in orderly fashion

Difficulty with cancellation occurs in situations where resources have been allocated to a canceled thread or where a thread is cancelled during updating data it is sharing (i.e., global variable)

This becomes troublesome with asynch cancellation

OS will reclaim system resources from cancelled thread but will not reclaim ALL

THEREFORE! may not free all necessary system-wide resource

```
pthread_t tid;

/* create the thread */
pthread_create(&tid, 0, worker, NULL);

. . .

/* cancel the thread */
pthread_cancel(tid);

/* wait for the thread to terminate */
pthread_join(tid, NULL);
```

Image 16: Simulating cancelling a thread

Invoking `pthread_cancel()` indicates only a request to cancel target thread HOWEVER **actual cancellation** depends if the target thread is set up to handle the request.

Mode	State	Type
Off	Disabled	–
Deferred	Enabled	Deferred
Asynchronous	Enabled	Asynchronous

Image 17: Modes, states and types

- If thread has cancellation disabled, cancellation remains pending until thread enables it
- **Default type is deferred**
 - Cancellation can only occur when thread reaches cancellation point

- cleanup_handler is invoked

Thread Local Storage

- Allows each thread to have its own copy of data
- Useful when you do not have control over thread creation process
- TLS is unique to each thread

Scheduler Activations

Both Many to Many and Two-level model require communication to maintain appropriate number of kernel threads in application

immediate data structure between user and kernel threads - lightweight process lwp

- Appears to be virtual processor
- each LWP attached to kernel thread
- how many lwps to create?

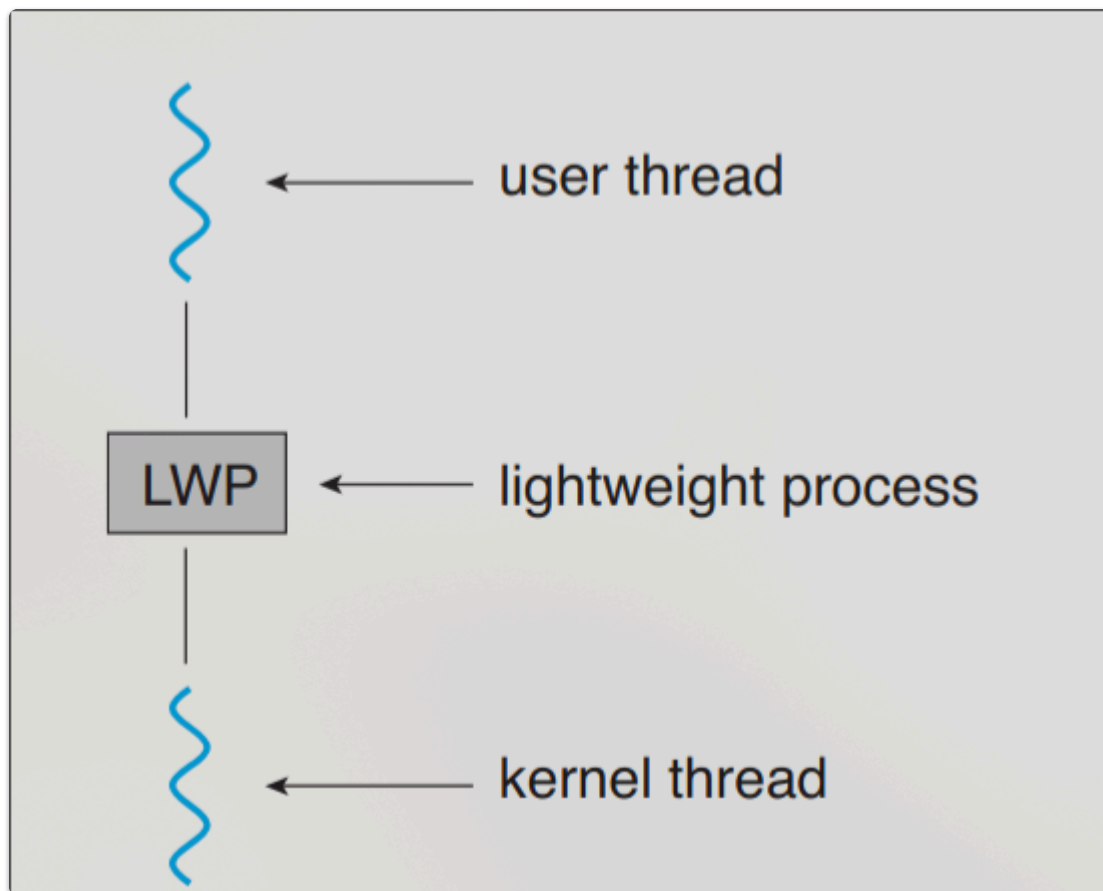


Image 18: LWP

Scheduler activations provide **upcalls** (communication mechanism) from kernel to **upcall handler**

Windows Threads

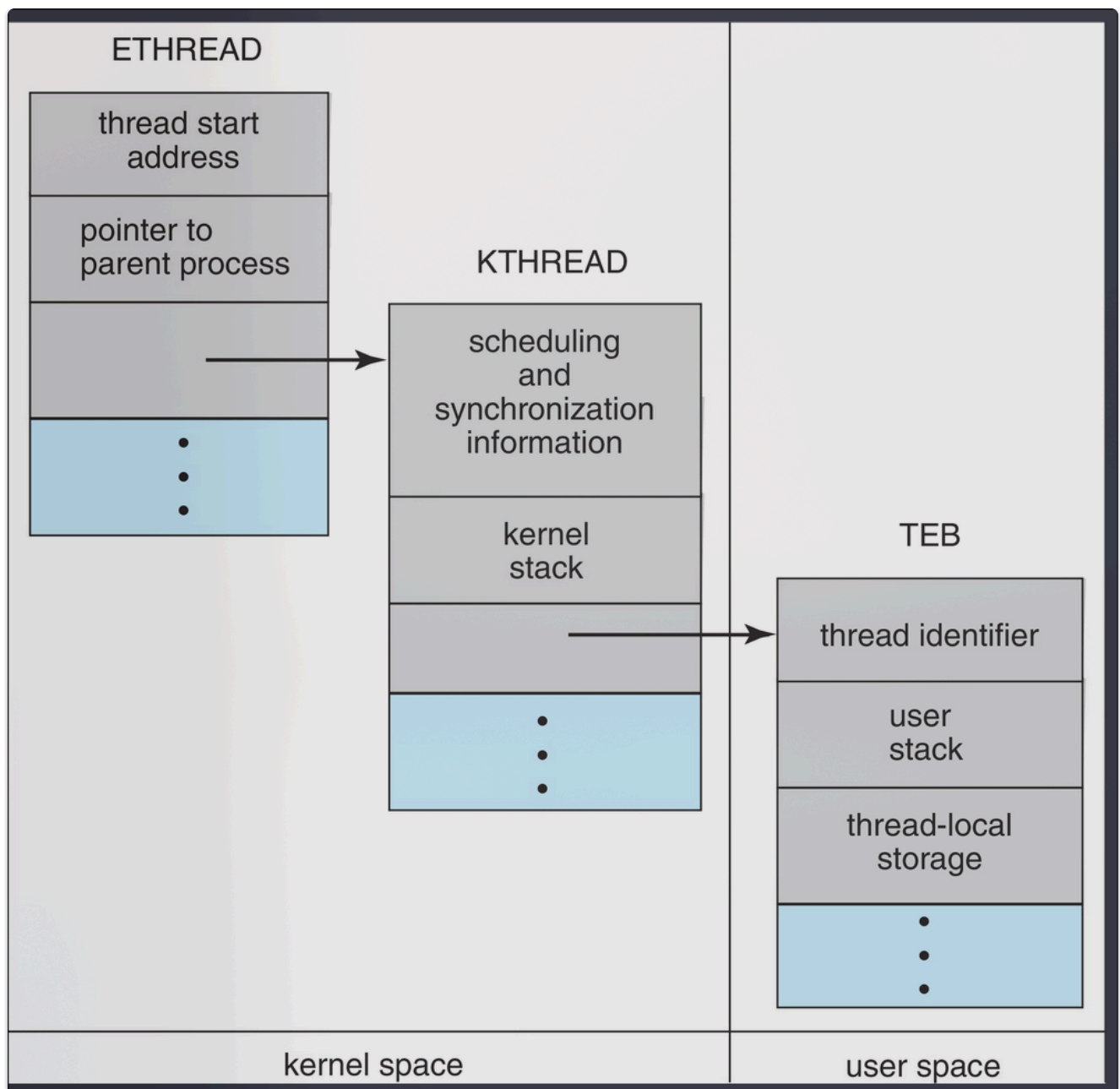
Windows API - primary API for Windows applications threading
one to one mapping, kernel-level

contains:

- thread id
- register set representing state of processor
- separate user and kernel stacks for when thread runs in user mode or kernel mode
- private data storage

register set, stacks and private storage

- The primary data structures of a thread include:
 - ETHREAD (executive thread block) – includes pointer to process to which thread belongs and to KTHREAD, in kernel space
 - KTHREAD (kernel thread block) – scheduling and synchronization info, kernel-mode stack, pointer to TEB, in kernel space
 - TEB (thread environment block) – thread id, user-mode stack, thread-local storage, in user space



Linux threads

Linux Threads

- Linux refers to them as **tasks** rather than threads
- Thread creation is done through clone() system call
- clone() allows a child task to share the address space of the parent task (process)
- Flags control behavior

flag	meaning
CLONE_FS	File-system information is shared.
CLONE_VM	The same memory space is shared.
CLONE_SIGHAND	Signal handlers are shared.
CLONE_FILES	The set of open files is shared.

- struct task_struct points to process data structures (shared or unique)